LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 6

DOUBLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Itrhoh Anggun Pamungkas

109082500117

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 5 : https://github.com/anggungun/struktur_data/tree/main/Modul6

GUIDE

1. Guide 1

dll.h

```
#ifndef DLL_H
#define DLL_H

#include <iostream>
#include <string>
using namespace std;

struct Kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};
struct ElmList {
    Kendaraan info;
    ElmList* next;
    ElmList* prev;
};

// STRUKTUR LIST
struct List {
    ElmList* first;
    ElmList* last;
};

// FUNGSI DASAR
void createList(List &L);
ElmList* alokasi(Kendaraan x);
void dealokasi(ElmList* &P);
void printInfo(List L);
ElmList* findElm(List L, string nopol);

// FUNGSI INSERT
void insertFirst(List &L, ElmList* P);
void insertLast(List &L, ElmList* P);
void insertAfter(List &L, ElmList* Prec, ElmList* P);
void insertBefore(List &L, ElmList* Succ, ElmList* P);

// FUNGSI DELETE
void deleteFirst(List &L, ElmList* &P);
```

```
void deleteLast(List &L, ElmList* &P);
void deleteAfter(List &L, ElmList* Prec, ElmList* &P);
void deleteBefore(List &L, ElmList* Succ, ElmList* &P);

#endif
```

dll.cpp

```
#include "dll.h"

// MEMBUAT LIST KOSONG
void createList(List &L) {
    L.first = nullptr;
    L.last = nullptr;
}

// MEMBUAT NODE BARU
ElmList* alokasi(Kendaraan x) {
    ElmList* P = new ElmList;

    P->info = x;
    P->next = nullptr;
    P->prev = nullptr;

    return P;
}

// MEMBEBASKAN MEMORI NODE
void dealokasi(ElmList* &P) {
    if (P != nullptr) {
        delete P;
        P = nullptr;
    }
}

// MENAMPILKAN SEMUA DATA KENDARAAN
void printInfo(List L) {
    ElmList* P = L.first;

    if (P == nullptr) {
        cout << "List kosong." << endl;
        return;
    }

    while (P != nullptr) {
        cout << "Nopol    : " << P->info.nopol << endl;
        cout << "Warna    : " << P->info.warna << endl;
        cout << "Tahun    : " << P->info.thnBuat << endl;
        cout << "--------------------" << endl;
```

```
        P = P->next;
    }
}

// MENCARI NODE BERDASARKAN NOMOR POLISI
ElmList* findElm(List L, string nopol) {
    ElmList* P = L.first;

    while (P != nullptr) {
        if (P->info.nopol == nopol) {
            return P;
        }

        P = P->next;
    }

    return nullptr;
}

// INSERT FIRST: MENAMBAHKAN NODE DI AWAL
void insertFirst(List &L, ElmList* P) {
    if (P == nullptr) {
        return;
    }

    P->prev = nullptr;
    P->next = L.first;

    if (L.first != nullptr) {
        L.first->prev = P;
    } else {
        L.last = P;
    }

    L.first = P;
}

// INSERT LAST: MENAMBAHKAN NODE DI AKHIR
void insertLast(List &L, ElmList* P) {
    if (P == nullptr) {
        return;
    }

    P->next = nullptr;
    P->prev = L.last;

    if (L.last != nullptr) {
```

```
        L.last->next = P;
    } else {
        L.first = P;
    }

    L.last = P;
}

// INSERT AFTER: MENAMBAHKAN NODE SETELAH NODE ACUAN
void insertAfter(List &L, ElmList* Prec, ElmList* P) {
    if (Prec == nullptr || P == nullptr) {
        return;
    }

    if (Prec == L.last) {
        insertLast(L, P);
        return;
    }

    P->next = Prec->next;
    P->prev = Prec;

    Prec->next->prev = P;
    Prec->next = P;
}

// INSERT BEFORE: MENAMBAHKAN NODE SEBELUM NODE ACUAN
void insertBefore(List &L, ElmList* Succ, ElmList* P) {
    if (Succ == nullptr || P == nullptr) {
        return;
    }

    if (Succ == L.first) {
        insertFirst(L, P);
        return;
    }

    P->next = Succ;
    P->prev = Succ->prev;

    Succ->prev->next = P;
    Succ->prev = P;
}

// DELETE FIRST: MENGELUARKAN NODE PERTAMA
void deleteFirst(List &L, ElmList* &P) {
    if (L.first == nullptr) {
        P = nullptr;
```

```
        return;
    }

    P = L.first;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.first = P->next;
        L.first->prev = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE LAST: MENGELUARKAN NODE TERAKHIR
void deleteLast(List &L, ElmList* &P) {
    if (L.last == nullptr) {
        P = nullptr;
        return;
    }

    P = L.last;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.last = P->prev;
        L.last->next = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE AFTER: MENGELUARKAN NODE SETELAH NODE ACUAN
void deleteAfter(List &L, ElmList* Prec, ElmList* &P) {
    if (Prec == nullptr || Prec->next == nullptr) {
        P = nullptr;
        return;
    }

    P = Prec->next;

    if (P == L.last) {
```

```
        deleteLast(L, P);
        return;
    }

    Prec->next = P->next;
    P->next->prev = Prec;

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE BEFORE: MENGELUARKAN NODE SEBELUM NODE ACUAN
void deleteBefore(List &L, ElmList* Succ, ElmList* &P) {
    if (Succ == nullptr || Succ->prev == nullptr) {
        P = nullptr;
        return;
    }

    P = Succ->prev;

    if (P == L.first) {
        deleteFirst(L, P);
        return;
    }

    Succ->prev = P->prev;
    P->prev->next = Succ;

    P->next = nullptr;
    P->prev = nullptr;
}
```

main.cpp

```
#include "dll.h"

int main() {
    List L;
    createList(L);

    Kendaraan K1 = {"D001", "Merah", 2022};
    Kendaraan K2 = {"D002", "Hitam", 2023};
    Kendaraan K3 = {"D003", "Biru", 2024};
    Kendaraan K4 = {"D004", "Abu-abu", 2025};
    Kendaraan K0 = {"D000", "Putih", 2021};

    // INSERT LAST: MEMASUKKAN D001 DI AKHIR
    ElmList* P = alokasi(K1);
```

```
    insertLast(L, P);

    // INSERT LAST: MEMASUKKAN D002 DI AKHIR
    P = alokasi(K2);
    insertLast(L, P);

    // INSERT FIRST: MEMASUKKAN D000 DI AWAL
    P = alokasi(K0);
    insertFirst(L, P);

    // INSERT AFTER: MEMASUKKAN D003 SETELAH D001
    ElmList* Prec = findElm(L, "D001");

    P = alokasi(K3);
    insertAfter(L, Prec, P);

    // INSERT BEFORE: MEMASUKKAN D004 SEBELUM D002
    ElmList* Succ = findElm(L, "D002");

    P = alokasi(K4);
    insertBefore(L, Succ, P);

    // MENAMPILKAN LIST SETELAH INSERT
    cout << "=== DATA SETELAH INSERT ===" << endl;
    printInfo(L);

    // FIND ELEMENT: MENCARI D001
    P = findElm(L, "D001");

    cout << "=== HASIL PENCARIAN ===" << endl;

    if (P != nullptr) {
        cout << "Nopol ditemukan: " << P->info.nopol << endl;
        cout << "Warna: " << P->info.warna << endl;
        cout << "Tahun: " << P->info.thnBuat << endl;
    } else {
        cout << "Kendaraan tidak ditemukan." << endl;
    }

    // DELETE AFTER: MENGHAPUS D003 SETELAH D001
    Prec = findElm(L, "D001");

    deleteAfter(L, Prec, P);
    dealokasi(P);

    // MENAMPILKAN LIST SETELAH DELETE
    cout << endl;
    cout << "=== DATA SETELAH DELETE AFTER D001 ===" << endl;
```

```
    printInfo(L);

    // MEMBERSIHKAN SISA NODE
    while (L.first != nullptr) {
        deleteFirst(L, P);
        dealokasi(P);
    }

    return 0;
}
```

Deskripsi program

Program ini mengimplementasikan Doubly Linked List data kendaraan (nopol, warna, tahun) yang dibagi menjadi dll.h, dll.cpp, dan main_1.cpp, dengan operasi insert (first, last, after, before), pencarian, delete (first, last, after, before), dan penampilan data. Di main, list dibangun menjadi D000, D001, D003, D004, D002, lalu D001 dicari, D003 dihapus dengan deleteAfter sehingga list menjadi D000, D001, D004, D002, dan semua node dibersihkan di akhir.

SOAL

1. **Soal 1**

Doublylist.h

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <string>
using namespace std;

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address first;
    address last;
};

void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);

#endif
```

Doublylist.cpp

```
#include <iostream>
#include "Doublylist.h"
using namespace std;

void createList(List &L) {
    L.first = NULL;
    L.last = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
        L.first = P;
        L.last = P;
    } else {
        P->prev = L.last;
        L.last->next = P;
        L.last = P;
    }
}

void printInfo(List L) {
    address P = L.first;

    while (P != NULL) {
        cout << "Nomor Polisi: " << P->info.nopol << endl;
        cout << "Warna: " << P->info.warna << endl;
        cout << "Tahun Buat: " << P->info.thnBuat << endl;
        cout << "--------------------" << endl;

        P = P->next;
    }
}
```

main.cpp

```
#include <iostream>
#include "Doublylist.h"
using namespace std;

int main() {
    List L;
    address P;
    infotype K;

    createList(L);

    K.nopol = "D001";
    K.warna = "Merah";
    K.thnBuat = 2020;
    P = alokasi(K);
    insertLast(L, P);

    K.nopol = "D002";
    K.warna = "Hitam";
    K.thnBuat = 2021;
    P = alokasi(K);
    insertLast(L, P);

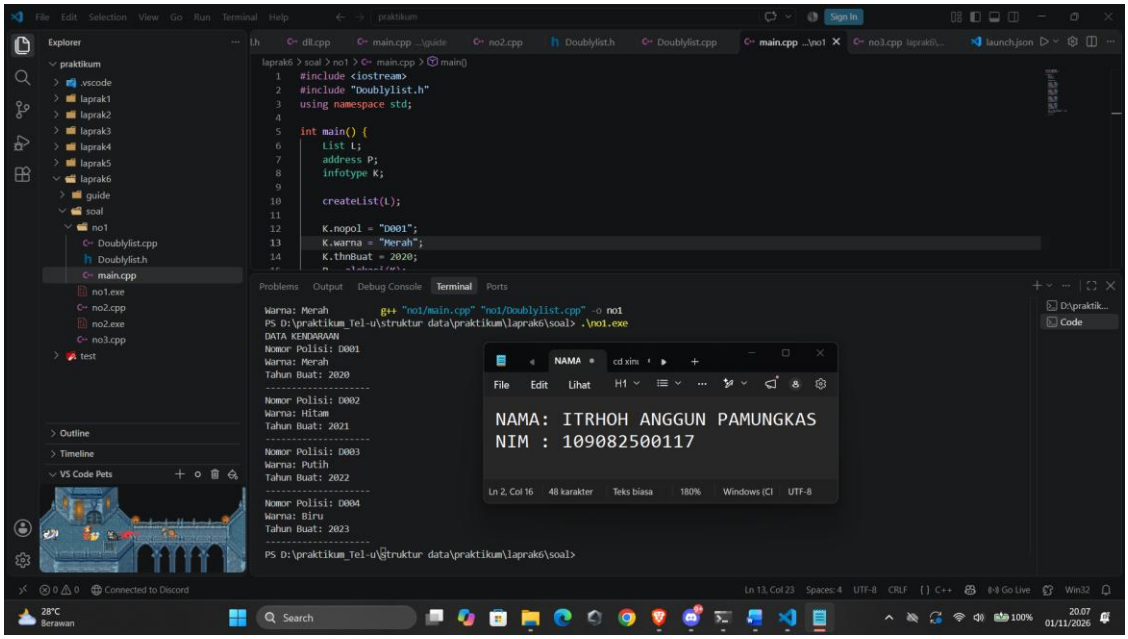
    K.nopol = "D003";
    K.warna = "Putih";
    K.thnBuat = 2022;
    P = alokasi(K);
    insertLast(L, P);

    K.nopol = "D004";
    K.warna = "Biru";
    K.thnBuat = 2023;
    P = alokasi(K);
    insertLast(L, P);

    cout << "DATA KENDARAAN" << endl;
    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Pada program ini, membuat ADT Doubly Linked List untuk menyimpan data kendaraan. Setiap kendaraan memiliki nomor polisi, warna, dan tahun pembuatan. Data dimasukkan menggunakan insertLast() sehingga kendaraan ditambahkan pada bagian akhir list.

2. Soal 2

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address first;
```

```
    address last;
};

void createList(List &L) {
    L.first = NULL;
    L.last = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
        L.first = P;
        L.last = P;
    } else {
        P->prev = L.last;
        L.last->next = P;
        L.last = P;
    }
}

void printInfo(List L) {
    address P = L.first;

    while (P != NULL) {
        cout << P->info.nopol << " - "
             << P->info.warna << " - "
             << P->info.thnBuat << endl;
        P = P->next;
    }
}

address findElm(List L, string x) {
    address P = L.first;

    while (P != NULL) {
        if (P->info.nopol == x) {
            return P;
        }

        P = P->next;
    }
```

```
    return NULL;
}

int main() {
    List L;
    address P, hasil;
    infotype K;

    createList(L);

    K = {"D001", "Merah", 2020};
    P = alokasi(K);
    insertLast(L, P);

    K = {"D002", "Hitam", 2021};
    P = alokasi(K);
    insertLast(L, P);

    K = {"D003", "Putih", 2022};
    P = alokasi(K);
    insertLast(L, P);

    K = {"D004", "Biru", 2023};
    P = alokasi(K);
    insertLast(L, P);

    cout << "DATA KENDARAAN:" << endl;
    printInfo(L);

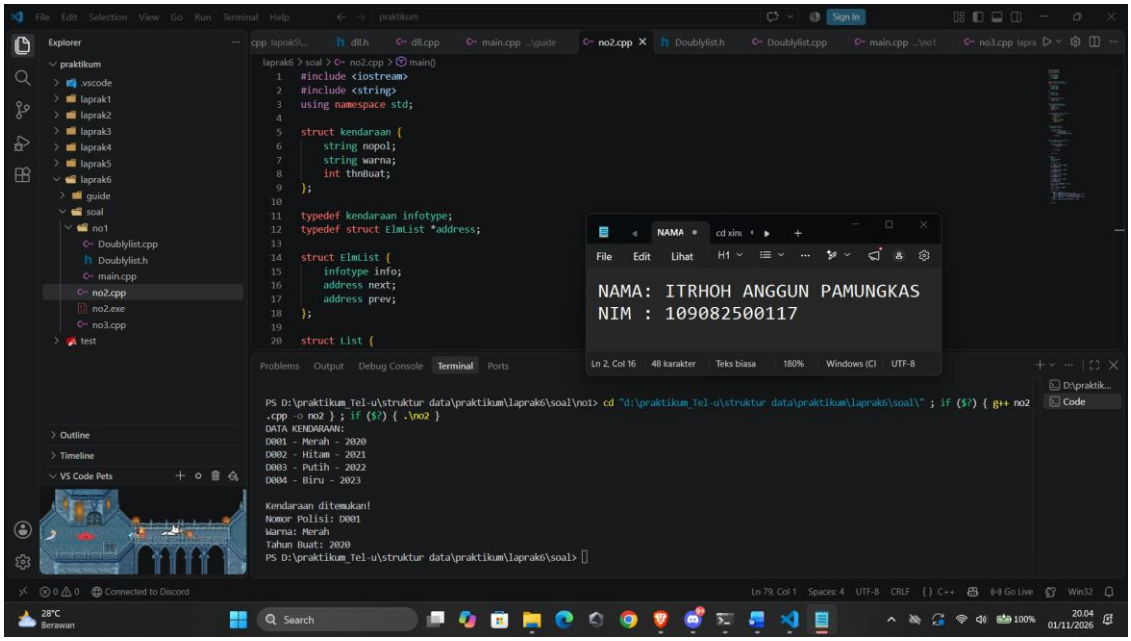
    hasil = findElm(L, "D001");

    cout << endl;

    if (hasil != NULL) {
        cout << "Kendaraan ditemukan!" << endl;
        cout << "Nomor Polisi: " << hasil->info.nopol <<
endl;
        cout << "Warna: " << hasil->info.warna << endl;
        cout << "Tahun Buat: " << hasil->info.thnBuat <<
endl;
    } else {
        cout << "Kendaraan tidak ditemukan." << endl;
    }

    return 0;
}
```

Screenshoot program



Deskripsi program

Pada nomor ini, membuat fungsi findElm() untuk mencari kendaraan berdasarkan nomor polisi. Program akan memeriksa nomor polisi setiap elemen dari awal list. Jika menemukan D001, program akan menampilkan data kendaraan tersebut.

3. Soal 3

Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
    address prev;
};

struct List {
    address first;
```

```
    address last;
};

void createList(List &L) {
    L.first = NULL;
    L.last = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

void insertLast(List &L, address P) {
    if (L.first == NULL) {
        L.first = P;
        L.last = P;
    } else {
        P->prev = L.last;
        L.last->next = P;
        L.last = P;
    }
}

void printInfo(List L) {
    address P = L.first;

    while (P != NULL) {
        cout << P->info.nopol << " - "
             << P->info.warna << " - "
             << P->info.thnBuat << endl;

        P = P->next;
    }
}

//menghapus elemen pertama
void deleteFirst(List &L, address &P) {
    if (L.first != NULL) {
        P = L.first;

        L.first = L.first->next;

        if (L.first != NULL) {
            L.first->prev = NULL;
```

```
            } else {
                L.last = NULL;
            }
            P->next = NULL;
            P->prev = NULL;
        }
    }

    //menghapus elemen terakhir
    void deleteLast(List &L, address &P) {
        if (L.last != NULL) {
            P = L.last;

            L.last = L.last->prev;

            if (L.last != NULL) {
                L.last->next = NULL;
            } else {
                L.first = NULL;
            }
            P->next = NULL;
            P->prev = NULL;
        }
    }

    //menghapus elemen setelah Prec
    void deleteAfter(List &L, address Prec, address &P) {
        if (Prec != NULL && Prec->next != NULL) {
            P = Prec->next;

            Prec->next = P->next;

            if (P->next != NULL) {
                P->next->prev = Prec;
            } else {
                L.last = Prec;
            }
            P->next = NULL;
            P->prev = NULL;
        }
    }

    int main() {
        List L;
        address P, hapus, cari;
        infotype K;

        createList(L);
```

```
    K = {"D001", "Merah", 2020};
    P = alokasi(K);
    insertLast(L, P);

    K = {"D002", "Hitam", 2021};
    P = alokasi(K);
    insertLast(L, P);

    K = {"D003", "Putih", 2022};
    P = alokasi(K);
    insertLast(L, P);

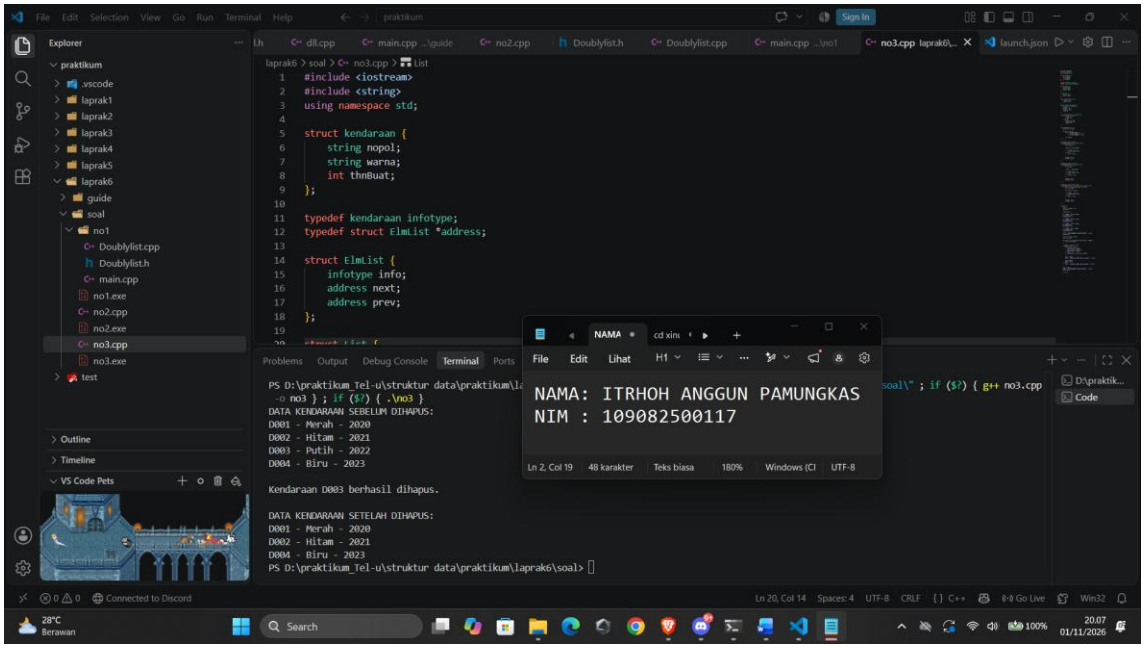
    K = {"D004", "Biru", 2023};
    P = alokasi(K);
    insertLast(L, P);

    cout << "DATA KENDARAAN SEBELUM DIHAPUS:" << endl;
    printInfo(L);

    //mencari kendaraan D003
    cari = L.first;
    while (cari != NULL && cari->info.nopol != "D003") {
        cari = cari->next;
    }
    //menghapus kendaraan D003
    if (cari != NULL) {
        if (cari == L.first) {
            deleteFirst(L, hapus);
        } else if (cari == L.last) {
            deleteLast(L, hapus);
        } else {
            deleteAfter(L, cari->prev, hapus);
        }
        cout << endl;
        cout << "Kendaraan D003 berhasil dihapus." << endl;
        delete hapus;
        hapus = NULL;
    } else {
        cout << "Kendaraan D003 tidak ditemukan." << endl;
    }

    cout << endl;
    cout << "DATA KENDARAAN SETELAH DIHAPUS:" << endl;
    printInfo(L);
    return 0;
}
```

Screenshoot program



Deskripsi program

Pada nomor ini, membuat tiga fungsi untuk menghapus elemen pada Doubly Linked List, yaitu deleteFirst(), deleteLast(), dan deleteAfter(). Program mencari kendaraan dengan nomor polisi D003. Karena D003 berada di tengah list, program menggunakan deleteAfter() untuk menghapus elemen tersebut.